

The **Web Storage API** is a built-in browser mechanism that allows web applications to **store key-value data directly in a user's browser**. It provides a much cleaner, higher-capacity

1. Web Storage API

The [MDN Web Storage API](#) provides a simple way for websites to save text data on the client side. It consists of two separate mechanisms:

- **localStorage**: Stores data permanently until it is explicitly cleared by code or by the user.
- **sessionStorage**: Stores data temporarily for a single browsing session.

Both mechanisms are strictly tied to the site's **origin** (a combination of protocol, domain name, and port number). A site can only access the data it created itself.

2. sessionStorage

[sessionStorage](#) is an object within the Web Storage API that isolates data to a **specific browser tab or window instance**.

- **Lifecycle**: The data lives as long as the tab or window stays open.
- **Persistence**: It survives page refreshes and restores, but it disappears completely the moment the user closes that specific tab.
- **Tab Isolation**: If you open the same website in two different tabs, each tab gets its own separate `sessionStorage` bucket.

3. getItem()

The `getItem()` function is a standard method used to **retrieve saved data** from either `sessionStorage` or `localStorage`.

- **Arguments**: It takes a single parameter: the name of the text `key` you want to look up.
- **Return Value**: It returns the saved value as a **string**. If the requested key does not exist in the browser's storage, it returns `null`.

•

Code Example: Putting It Together

Because Web Storage only stores strings, you will often use `JSON.stringify()` to save complex data types like arrays or objects, and `JSON.parse()` when you retrieve them.

javascript

```
// 1. Saving data using setItem()
sessionStorage.setItem("userRole", "admin");

const userObject = { id: 101, name: "Alex" };
sessionStorage.setItem("profile", JSON.stringify(userObject));

// 2. Retrieving data using getItem()
const role = sessionStorage.getItem("userRole");
console.log(role); // Output: "admin"

// 3. Retrieving and parsing a complex object
const savedProfile = sessionStorage.getItem("profile");
if (savedProfile) {
    const user = JSON.parse(savedProfile);
    console.log(user.name); // Output: "Alex"
}

// 4. Looking up a non-existent item
const missingItem = sessionStorage.getItem("fakeKey");
console.log(missingItem); // Output: null
```